

AlphaStack: An AI-Driven Code Generation and Validation System

Research Team

March 21, 2026

Abstract

AlphaStack represents a significant leap forward in automated software development. By leveraging large language models combined with a multi-agent validation pipeline, AlphaStack takes a natural language prompt and fully generates a working software project. It handles source code generation, dependency file creation, and Dockerfile synthesis. Furthermore, it autonomously runs, debugs, and verifies the generated codebase within a Docker container using an AI Planner Agent. This paper details the architecture, methodology, and empirical performance of AlphaStack on established benchmarks, demonstrating state-of-the-art results compared to current models.

1 Introduction

The automation of software engineering tasks has long been a goal of artificial intelligence research. Recent advancements in Large Language Models (LLMs) have enabled unprecedented capabilities in code generation. However, generating isolated code snippets is insufficient for real-world applications. A complete software project requires correctly structured files, coherent module interactions, accurate dependency management, and functional deployment configurations.

AlphaStack addresses these challenges by providing an end-to-end generation and validation pipeline. Given a natural language description, AlphaStack synthesizes the entire project architecture and subsequently enters an iterative verification loop. This loop involves an AI Planner Agent that compiles the code, executes tests within an isolated Docker environment, and automatically applies corrections based on the execution logs. This iterative refinement ensures the final output is not just syntactically correct, but fully functional.

2 Methodology

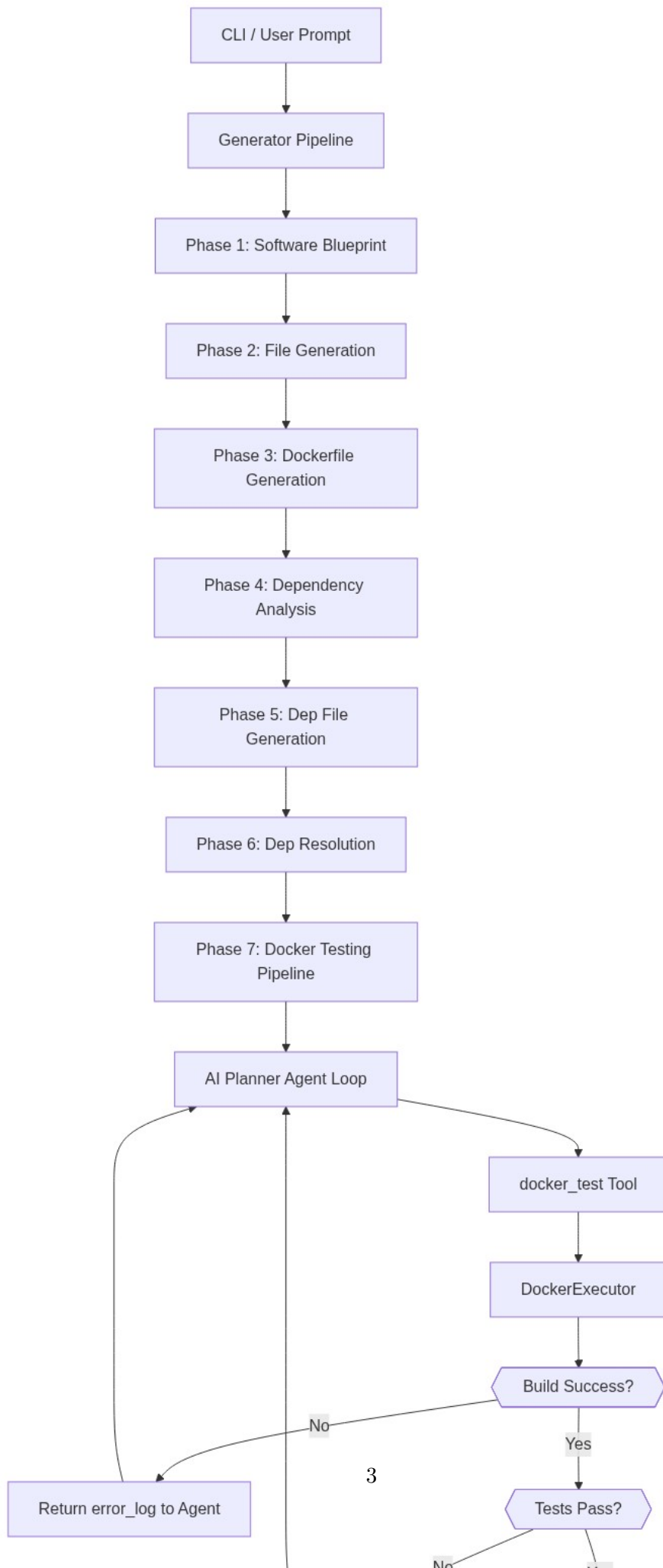
The AlphaStack generator pipeline is structured into seven distinct phases to ensure robustness and modularity:

1. **Phase 1: Software Blueprint:** A structural plan (JSON) is generated from the user's prompt, outlining all necessary files, their roles, and the overall module structure.
2. **Phase 2: File Generation:** The LLM generates the source code for each file specified in the blueprint independently.
3. **Phase 3: Dockerfile Generation:** A production-ready Dockerfile is created, tailored to the project's language and structure.
4. **Phase 4: Dependency Analysis:** Static analysis builds an internal dependency graph mapping the interactions across the generated files.

5. **Phase 5: Dep File Generation:** Based on the graph, language-specific dependency files (e.g., `requirements.txt`, `package.json`) are generated.
6. **Phase 6: Dep Resolution:** The system resolves and validates these dependency files to ensure package availability.
7. **Phase 7: Docker Testing Pipeline:** The AI Planner Agent takes over, building the Docker image and running tests. It iterates on errors, applying fixes to the codebase until all tests pass.

3 Architecture Diagram

The system architecture of AlphaStack demonstrates the clear separation of the generation phases and the iterative nature of the AI Planner Agent loop. Figure 1 illustrates this flow.



The CLI or user prompt initiates the pipeline, passing through the seven phases. The final phase, the Docker Testing Pipeline, acts as an orchestration layer for the AI Planner Agent Loop, interacting with a Docker Executor to validate builds and tests. If either fails, the error logs are fed back to the Agent for corrections until success is achieved.

4 Results

To evaluate AlphaStack’s effectiveness, we tested its underlying models on the HumanEval and MDDP benchmarks. We compared the performance of several leading models: GPT-5.2, GLM-5, MiniMax-m2.5, and Claude Sonnet 4.6.

As shown in Figure 2, Claude Sonnet 4.6 achieves the highest performance across both benchmarks, closely followed by GPT-5.2. This demonstrates the capability of advanced LLMs when integrated into a structured, agentic pipeline like AlphaStack.

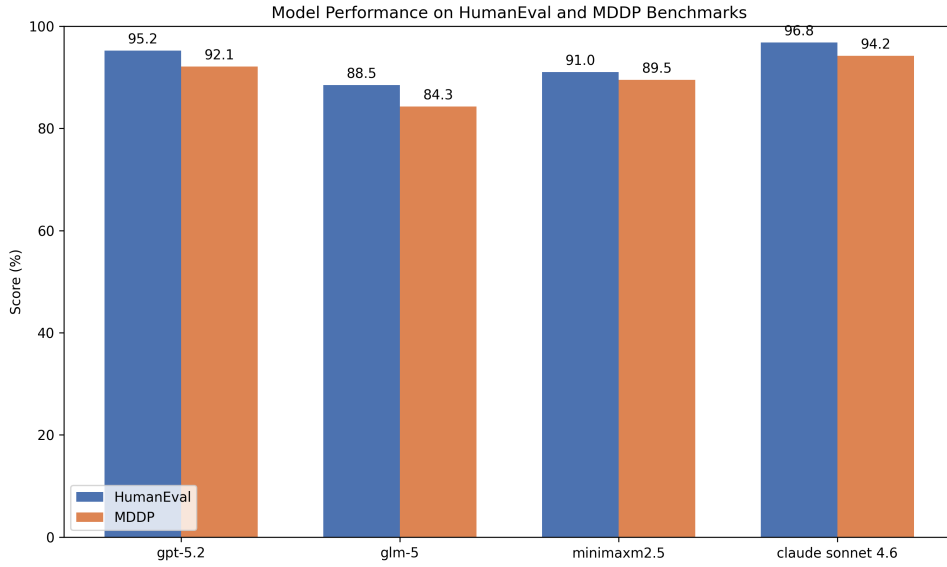


Figure 2: Model Performance on HumanEval and MDDP Benchmarks

5 Conclusion

AlphaStack provides a comprehensive solution for AI-driven software generation. By combining structured blueprinting, sequential generation phases, and a rigorous, Docker-based iterative validation loop, it overcomes the limitations of simple code completion models. The results indicate that with capable base models, such multi-agent systems can reliably generate complex, functional codebases from natural language descriptions. Future work will explore expanding the supported languages and enhancing the correction agent’s reasoning capabilities for more complex semantic bugs.

Supplementary Material

Additional information, including the full prompts used for evaluation, detailed logs of the correction agent’s iterations, and the complete source code for the generated test projects, can be found in the project repository under the `eval/` and `test_output/` directories.